

Utility Analysis Dashboard

Technical Design Note

Generated 2026-03-10 · Internal reference

1. Stack

Runtime: Python 3.11+, Streamlit, Plotly (go + px), pandas / numpy

Venv: shared ../finance_vis/venv_finance — do not create a new one

Launch:

```
source ../finance_vis/venv_finance/bin/activate && streamlit run app.py
```

Input: Korean .xslm/.xlsx files loaded from ./data/ (configurable in sidebar)

2. Module Map

```
app.py          <- Page config, top-level routing, file loading
sidebar.py      <- File loader (dir scan -> _FileEntry list), bins/tail stubs
data.py         <- Excel I/O (@st.cache_data), sheet name constants, billing period
features.py     <- Column engineering: create_change_columns, build_from_two_files,
                  aggregate_by_brand, split_brand_by_floor, floor parsing helpers
filters.py      <- render_meter_filters, show_filter_widgets, apply_sheet_filter
viz.py          <- plot_hist_with_tails (Plotly go.Bar histogram, IQR shading)
lang.py         <- t() translation helper (Korean only, key->string)

meter_view.py   <- 검침내역 full pipeline: load -> filter -> histograms -> 5 tabs
summary.py      <- 총 유틸리티 순위 + 면적당 총비용
billing.py      <- 수도광열비 부과 내역 view
ehp.py          <- EHP(OAC)검침자료 view
water.py / hotwater.py / electricity.py <- per-sheet views
brand_profile.py <- Brand profile tab with peer comparison charts

tab_anomaly.py  <- 이상감지 tab (composite score, heatmap, spike/cost/HVAC sub-tabs)
tab_cross.py    <- 비용분석 tab (unit costs, electricity breakdown)
tab_efficiency.py <- 효율분석 tab (per-m2 usage benchmarking)
tab_corr.py / tab_reconciliation.py <- correlation & reconciliation tabs

anomaly_features.py <- build_anomaly_df: meter+billing+elec+water -> anomaly scores
cross_features.py <- build_unit_costs, build_elec_breakdown
report.py       <- generate_report_pdf (검침내역 main report)
biz_report.py   <- generate_anomaly_pdf, generate_cross_pdf, generate_efficiency_pdf
billing_report.py / ehp_report.py / hvac_report.py <- per-sheet PDF generators
```

3. Navigation Architecture

```
sac.tabs (sidebar, left-position) -> nav_mode
+-- 시트 보기 -> sheet selectbox -> billing/ehp/water/hotwater/electricity/meter_view
+-- 분석      -> analysis selectbox
|   +-- 요약 분석 -> render_summary_view(water_df, hw_df, el_df)
|   +-- 업체 분석 -> render_meter_filters -> [이상감지|비용분석|효율분석] tabs
+-- 브랜드 프로필 -> render_brand_profile_tab
```

Key quirk: `sac.tabs` must render directly inside with `st.sidebar`: — never inside `st.empty()`. The empty placeholder causes the custom component to re-initialize on every rerun, losing state.

4. Data Pipeline (검침내역)

```
read_sheet()          # raw Excel, header=[2, 3, 4]
+- apply_header_rows() # flatten MultiIndex headers -> named columns
+- build_from_two_files() # prev file -> *_previous = prev month usage
+- create_change_columns() # *_change = curr-prev, *_pct = change/prev*100
+- aggregate_by_brand() # group by (brand, building), sum usage
+- split_brand_by_floor() # divide totals equally by floor count
```

Cumulative vs usage: Meter readings in Excel are cumulative. `build_from_two_files` renames them to `*_meter_curr/prev` and uses the pre-computed usage column (`water_usage_m3` etc.) as the actual current-month consumption.

Column Naming Convention

Column	Meaning
water_previous / water_current	This month's usage from prev/cur file
water_change	current - previous
water_pct	change / previous x 100
water_meter_prev / water_meter_curr	Original cumulative readings (kept for backward detection)

5. Floor Logic

Floor values are compound strings: "1F/2F", "2F~5F", "B2F/B1F".

Function	Purpose
<code>parse_floor_value(s)</code>	Returns list of individual floor strings from compound value
<code>get_simple_floors(df)</code>	Sorted unique floors for multiselect widget
<code>split_brand_by_floor(df, sel_floors)</code>	Divides brand totals equally by count of matched floors

When `sel_floors == ["All"]` and `sel_bldg == ["All"]` -> no splitting, use aggregate totals.

6. Histogram System (viz.py: `plot_hist_with_tails`)

Signature (key params)

```
plot_hist_with_tails(s, bins, lo, hi, title,
  source_df=None, val_col=None, key="hist",
  display_cols=None, tail_pct=None, val_scale=1.0)
```

- `lo / hi` must be in the same units as the plotted series `s`
- `val_scale` bridges when `s` is scaled (e.g. `/1e4` for 만원) but `source_df[val_col]` is raw 원:

```
mask = (source_df[val_col] / val_scale >= x0) & (source_df[val_col] / val_scale <= x1)
```

- Style locked: blue normal bars (#4C72B0), amber tail bars (#DD8A00), red median line (#C44E52), white bg, height=380px
- Outlier table: when source_df provided, auto-shows 이상치 목록 expander with all rows outside [lo, hi]

IQR Detection Pattern

```
q1, q3 = s.quantile(0.25), s.quantile(0.75)
iqr = q3 - q1
lo = q1 - k * iqr # k slider: 0.5-3.0, default 1.5, step 0.25
hi = q3 + k * iqr
# Show LaTeX:
st.markdown(
    f"$Q_1={q1:,.0f}, Q_3={q3:,.0f}, IQR={iqr:,.0f}$\\n\\n"
    f"$\\text{Lower}={lo:,.0f}, \\text{Upper}={hi:,.0f}; (k={k})$"
)
```

7. Money Formatting (원 units)

```
def _fmt_won(v):
    if abs(v) >= 1e8: return f"{v/1e8:,.0f} 억원"
    if abs(v) >= 1e4: return f"{v/1e4:,.0f} 만원"
    return f"{v:,.0f} 원"

# For bar charts -- compute _div from max(series):
_div, _unit = (1e8, "억원") if _max >= 1e8 else (1e4, "만원")
_xv = series / _div
# pass val_scale=_div to plot_hist_with_tails so bin-click filter still works
```

Rule: no decimals on 원 values — always :.0f.

8. Anomaly Scoring (anomaly_features.py)

Composite score = weighted sum of 5 components, each normalized to [0, 1]:

Component	Weight	Signal	Source sheet
금등	30%	SpikeMoM % change vs thresholds (100/50/20%)	검침내역
소비	25%	ConsQ-median	검침내역
비용	25%	Cost Z-scores of W/m3, W/kWh, 10kW/m2	수도광열비 부과 내역
HVAC	10%	kWh/m2 IQR-normalized	전체 전기 사용내역
일관성	10%	Consistency of zero-usage utilities	검침내역 +

Risk levels: 위험 (red) ≥ 0.65 · 주의 (orange) ≥ 0.40 · 관찰 (yellow) ≥ 0.20 · 정상 (green) < 0.20

9. PDF Generation Pattern (all biz tabs)

```
_pdf_key = f"{tab}_pdf_{file_name}"
_col_gen, _col_dl = st.columns([1, 2])
with _col_gen:
    if st.button("PDF 리포트 생성", key=f"gen_{tab}_pdf_{file_name}"):
        with st.spinner("폴 생성 중..."):
```

```
st.session_state[_pdf_key] = generate_X_pdf(df)
if _pdf_key in st.session_state:
    with _col_dl:
        st.download_button("다운로드", st.session_state[_pdf_key], ...)
```

Generate-then-cache pattern: PDF bytes stored in session_state to avoid re-generating on every Streamlit rerun. Generate and download buttons are separate widgets in adjacent columns.

10. Common Pitfalls

#	Issue	Fix
1	Closure bug in loops	Default arg binding: <code>def fn(key, _p=p, _cc=cc):</code> — never close over loop vars directly
2	sac.tabs state loss	Never wrap in <code>st.empty()</code> . Use <code>with st.sidebar:</code>
3	Bin-click scale mismatch	When series is scaled, pass <code>val_scale=factor</code> to <code>val</code>
4	Radio option ordering	Histogram always first: <code>["히스토그램", ...]</code> or <code>["히스토그램", ...]</code>
5	Decimals on <code>val</code>	Always <code>val:0f</code> , never <code>val:.2f</code>
6	EHP column parsing	Cumulative cols M-DG (0-based 12-110); stop at next section header; do NOT merge two tables
7	sac.tabs ValueError	Stored index $\geq$ tab count. Clear <code>nav_{file_name}</code> session state key

11. Notable Session State Keys

Key	Purpose
<code>nav_{file_name}</code>	Sidebar nav tab index (sac.tabs)
<code>anomaly_loaded_{file_name}</code>	Gate: has <code>이상감지</code> analysis been started?
<code>cross_loaded_{file_name}</code>	Gate: has cross-sheet data been loaded?
<code>anomaly_pdf_{file_name}</code>	Cached anomaly PDF bytes
<code>cross_pdf_{file_name}</code>	Cached cross-tab PDF bytes
<code>efficiency_pdf_{file_name}</code>	Cached efficiency PDF bytes
<code>{prefix}_iqr_k</code>	IQR k multiplier per histogram
<code>{prefix}_bins / {prefix}_bins_i</code>	Bins slider/input sync pair

12. Pending / Known Issues

- 면적당 총비용 unit scaling: Should display as 만원/m2 (not bare 만원 or 원/m2). Reverted at user request — ready to re-implement cleanly.
- No other outstanding tasks as of 2026-03-10.